

API HW

Cynthia Budiyanto

On Your Own - OMDb

1. Build a data set for a collection of movies by completing the **FILL IN** sections in the code below:

```
# http://www.omdbapi.com/?t=ANATOMY+OF+A+FALL
# http://www.omdbapi.com/?t=Death+Bed%3A+The+Bed+That+Eats
# http://www.omdbapi.com/?t=1998
# http://www.omdbapi.com/?t=Extremely+Loud+%26+Incredibly+Close
# http://www.omdbapi.com/?t=Fast+%26+Furious+Presents%3A+Hobbs+%26+Shaw

movies <- c("http://www.omdbapi.com/?t=ANATOMY+OF+A+FALL&apikey=", "http://www.omdbapi.com/?")

# Set up empty tibble
omdb <- tibble(Title = character(), Rated = character(), Genre = character(),
               Actors = character(), Metascore = character(), imdbRating = character(),
               BoxOffice = character())

# Use for loop to run through API request process 5 times,
#   each time filling the next row in the tibble
#   - can do max of 1000 GETs per day
for(i in 1:5) {
  url <- str_c(movies[i], "e39b58ad")
  Sys.sleep(0.5)
  onemovie <- GET(url)
  details <- content(onemovie, "parse")
  omdb[i,1] <- details$Title
  omdb[i,2] <- details$Rated # rating
  omdb[i,3] <- details$Genre # genres (single string - could be more than one)
  omdb[i,4] <- details$Actors # actors (single string - could be more than one)
  omdb[i,5] <- details$Metascore # metascore (be sure it is numeric)
```

```

omdb[i,6] <- details$imdbRating # imdb rating (be sure it is numeric)
omdb[i,7] <- details$BoxOffice  # box office (be sure it is numeric)
}

```

```
library(janitor)
```

```

omdb |>
  clean_names() |>
  mutate(metascore = parse_number(metascore),
         imdb_rating = parse_number(imdb_rating),
         box_office = parse_number(box_office))

```

```
# A tibble: 5 x 7
```

	title	rated	genre	actors	metascore	imdb_rating	box_office
	<chr>	<chr>	<chr>	<chr>	<dbl>	<dbl>	<dbl>
1	Anatomy of a Fall	R	Crim~	Sandr~	86	7.6	5076369
2	Death Bed: The Bed That E~	Not ~	Horrr~	Demen~	NA	4.3	NA
3	1998	N/A	Horrr~	Jason~	NA	NA	NA
4	Extremely Loud & Incredib~	PG-13	Adve~	Thoma~	46	6.9	31847881
5	Fast & Furious Presents: ~	PG-13	Acti~	Dwayn~	60	6.4	173956935

```

# could use stringr functions to further organize this data - separate
# different genres, different actors, etc. But don't need to now.

```

Navigating API documentation using httr

Navigate to the [Census API user guide](#) and click on the “Example API Queries” tab.

Let’s look at the Population Estimates Example and the American Community Survey (ACS) Example. These examples walk us through the steps to incrementally build up a URL to obtain desired data. This URL is known as a web API **request**.

https://api.census.gov/data/2019/acs/acs1?get=NAME,B02015_009E,B02015_009M&for=state:*

- <https://api.census.gov>: This is the **base URL**.
 - <http://>: The **scheme**, which tells your browser or program how to communicate with the web server. This will typically be either [http:](http://) or [https:](https://).
 - api.census.gov: The **hostname**, which is a name that identifies the web server that will process the request.
- [data/2019/acs/acs1](https://api.census.gov/data/2019/acs/acs1): The **path**, which tells the web server how to get to the desired resource.

- In the case of the Census API, this locates a desired dataset in a particular year.
- Other APIs allow search functionality. (e.g., News organizations have article searches.) In these cases, the path locates the search function we would like to call.
- `?get=NAME,B02015_009E,B02015_009M&for=state:*`: The **query parameters**, which provide the parameters for the function you would like to call.
 - We can view this as a string of key-value pairs separated by `&`. That is, the general structure of this part is `key1=value1&key2=value2`.

key	value
get	NAME,B02015_009E,B02015_009M
for	state:*

Typically, each of these URL components will be specified in the API documentation. Sometimes, the scheme, hostname, and path (`https://api.census.gov/data/2019/acs/acs1`) will be referred to as the **endpoint** for the API call.

On Your Own - Census

2. Adapt the code in `hennepin_tidycensus` to write a function called `MN_tract_data` to give the user choices about year, county, and variables to pull off. Show that `MN_tract_data(year = 2021, county = "Hennepin", variables = c("B01003_001", "B19013_001"))` works as expected. Make sure it also works for other years, counties, and variables (e.g. B25077_001 is median home price and B02001_002 is number of white residents).

```
MN_tract_data <- function(year, county, variables) {
  tidycensus::get_acs(
    year = year,
    state = "MN",
    geography = "tract",
    variables = variables,
    output = "wide",
    geometry = TRUE,
    county = county,
    show_call = TRUE
  )
}

MN_tract_data(year = 2021, county = "Hennepin", variables = c("B01003_001", "B19013_001"))
```

	0%
=	2%
==	3%
===	5%
====	6%
=====	8%
=====	10%
=====	11%
=====	13%
=====	15%
=====	16%
=====	18%
=====	19%
=====	20%
=====	22%
=====	24%
=====	26%
=====	27%
=====	29%
=====	30%
=====	33%

	=====	34%
	=====	36%
	=====	37%
	=====	38%
	=====	40%
	=====	41%
	=====	43%
	=====	45%
	=====	46%
	=====	48%
	=====	49%
	=====	51%
	=====	53%
	=====	54%
	=====	56%
	=====	57%
	=====	59%
	=====	61%
	=====	63%
	=====	64%
	=====	66%

=====	68%
=====	69%
=====	71%
=====	72%
=====	74%
=====	76%
=====	77%
=====	79%
=====	80%
=====	82%
=====	83%
=====	85%
=====	87%
=====	88%
=====	90%
=====	91%
=====	93%
=====	95%
=====	96%
=====	98%
=====	99%
=====	100%

Simple feature collection with 329 features and 6 fields

Geometry type: MULTIPOLYGON

Dimension: XY

Bounding box: xmin: -93.76838 ymin: 44.78538 xmax: -93.17722 ymax: 45.24662

Geodetic CRS: NAD83

First 10 features:

	GEOID		NAME	B01003_001E
1	27053024300	Census Tract 243, Hennepin County, Minnesota		4744
2	27053110500	Census Tract 1105, Hennepin County, Minnesota		4969
3	27053024006	Census Tract 240.06, Hennepin County, Minnesota		2205
4	27053022801	Census Tract 228.01, Hennepin County, Minnesota		2481
5	27053026908	Census Tract 269.08, Hennepin County, Minnesota		6139
6	27053025401	Census Tract 254.01, Hennepin County, Minnesota		4428
7	27053108600	Census Tract 1086, Hennepin County, Minnesota		2947
8	27053026824	Census Tract 268.24, Hennepin County, Minnesota		4551
9	27053106000	Census Tract 1060, Hennepin County, Minnesota		3375
10	27053000102	Census Tract 1.02, Hennepin County, Minnesota		4896
	B01003_001M	B19013_001E	B19013_001M	geometry
1	481	72240	5745	MULTIPOLYGON (((-93.31881 4...
2	651	80157	5307	MULTIPOLYGON (((-93.22237 4...
3	270	143125	22624	MULTIPOLYGON (((-93.35044 4...
4	359	133958	34619	MULTIPOLYGON (((-93.34793 4...
5	792	110246	3614	MULTIPOLYGON (((-93.39145 4...
6	648	68711	11097	MULTIPOLYGON (((-93.28347 4...
7	587	57470	15799	MULTIPOLYGON (((-93.24995 4...
8	483	127819	26964	MULTIPOLYGON (((-93.36073 4...
9	622	23492	5316	MULTIPOLYGON (((-93.25966 4...
10	597	59750	11634	MULTIPOLYGON (((-93.29919 4...

3. Use your function from (2) along with `map` and `list_rbind` to build a data set for Rice county for the years 2019-2021. Use your scraped data to plot trends in income over time and population over time.

```
years <- 2019:2021
```

```
rice19_21 <- map(years, ~MN_tract_data(year = .x, county = "Rice", variables = c("B01003_001E", "B19013_001E"),  
  list_rbind() |>  
  as_tibble() |>  
  rename(population = B01003_001E,  
    median_income = B19013_001E))
```

	0%
	2%
=	
==	4%
===	5%
====	7%
=====	9%
=====	11%
=====	13%
=====	14%
=====	16%
=====	18%
=====	20%
=====	22%
=====	23%
=====	25%
=====	27%
=====	29%
=====	31%
=====	32%
=====	34%
=====	36%

=====	38%
=====	40%
=====	41%
=====	43%
=====	45%
=====	47%
=====	48%
=====	50%
=====	52%
=====	54%
=====	56%
=====	57%
=====	59%
=====	61%
=====	63%
=====	65%
=====	66%
=====	68%
=====	70%
=====	72%
=====	74%
=====	75%

=====		77%
=====		79%
=====		81%
=====		83%
=====		84%
=====		86%
=====		88%
=====		90%
=====		92%
=====		93%
=====		95%
=====		97%
=====		99%
=====		100%

		0%
=		2%
==		3%
===		5%
====		6%
=====		8%

=====		9%
=====		11%
=====		12%
=====		14%
=====		15%
=====		17%
=====		18%
=====		20%
=====		21%
=====		23%
=====		24%
=====		26%
=====		27%
=====		29%
=====		30%
=====		32%
=====		33%
=====		35%
=====		36%
=====		38%
=====		39%

=====	41%
=====	42%
=====	44%
=====	45%
=====	47%
=====	48%
=====	50%
=====	51%
=====	53%
=====	54%
=====	56%
=====	57%
=====	59%
=====	60%
=====	62%
=====	63%
=====	65%
=====	66%
=====	68%
=====	69%
=====	71%
=====	72%

=====		74%
=====		75%
=====		77%
=====		78%
=====		80%
=====		81%
=====		83%
=====		84%
=====		86%
=====		87%
=====		89%
=====		90%
=====		92%
=====		93%
=====		95%
=====		96%
=====		98%
=====		99%
=====		100%

```
year <- tibble(c(year = map_dbl(1:13, ~2019),
                  year = map_dbl(14:29, ~2020),
                  year = map_dbl(30:45, ~2021))) |>
```

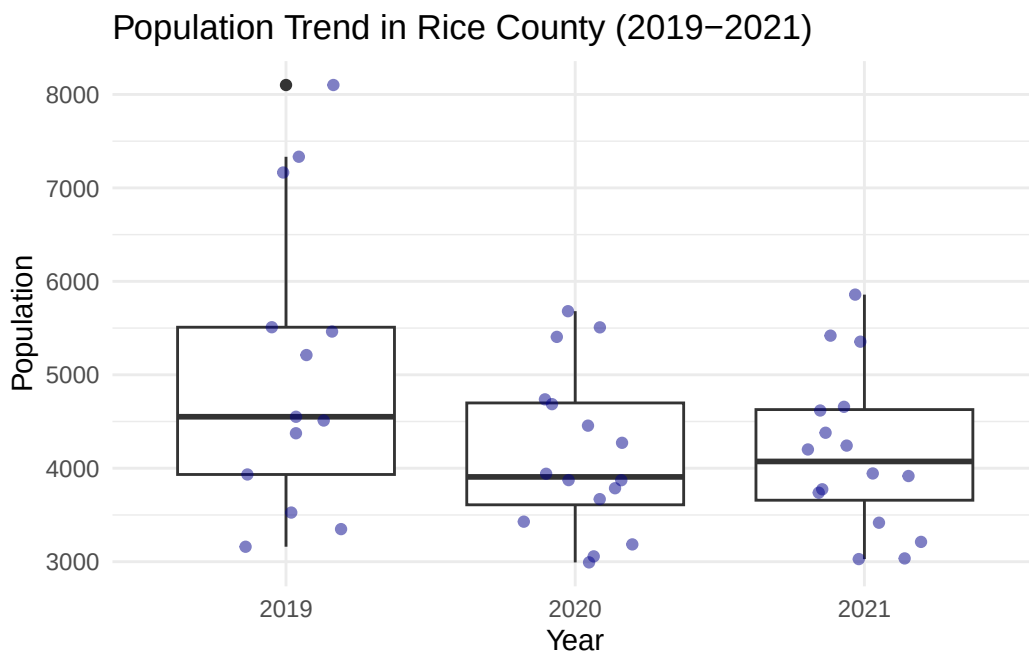
```

  rename("year" = `c(...)`)

ricedf<- rice19_21 |>
  select(population, median_income) |>
  bind_cols(year) |>
  mutate(year = as.character(year))

ggplot(ricedf, aes(x = year, y = population)) +
  geom_boxplot() +
  geom_jitter(width = 0.2, color = "darkblue", alpha = 0.5) +
  theme_minimal() +
  labs(
    title = "Population Trend in Rice County (2019-2021)",
    x = "Year",
    y = "Population"
  )

```



```

ggplot(ricedf, aes(x = year, y = median_income)) +
  geom_boxplot() +
  geom_jitter(width = 0.2, color = "darkblue", alpha = 0.5) +
  theme_minimal() +
  labs(

```

```
title = "Income Trend in Rice County (2019-2021)",  
x = "Year",  
y = "Income"  
)
```

